

PS2 — Hangman

Source: MIT OpenCourseWare, 6.100L *Introduction to CS and Programming Using Python*, Fall 2022, Problem Set 2. Adapted for this course under the Creative Commons **CC BY-NC-SA 4.0** license (<https://ocw.mit.edu/terms>). This adaptation is likewise shared under CC BY-NC-SA 4.0.

1) Assignment Overview

Item	Value
Original assignment	MIT 6.100L Fall 2022, Problem Set 2: Hangman
Topic	Writing and calling functions; loops until a condition is reached
Assigned week	Week 4
Files provided	<code>hangman.py</code> , <code>test_ps2_student.py</code> , <code>words.txt</code> (from the original OCW code zip) — save them all in the same directory
Point split	Problem 1.1 15% · 1.2 15% · 1.3 15% · Problem 2 (game incl. help) 55% (4 problems)

Objectives

- Writing and calling functions in Python
- Using loop mechanisms to repeat a computational process until a condition is reached

File Setup. Run `hangman.py` before writing any code to ensure your files are saved correctly. The starter code loads words from the `words.txt` file. You should see the following output in your shell:

```
Loading word list from file...
55900 words loaded
```

Game Overview. You will implement a function called `hangman` that will allow the user to play hangman against the computer. The computer picks the word, and the player tries to guess letters in the word.

1. The computer selects a word at random from the list of words in `words.txt` (all lowercase letters).
2. The user is given a certain number of guesses at the beginning.
3. The user inputs their guess, and the computer either:
 - reveals the letter if it exists in the secret word,
 - informs the user if their guess is invalid (i.e. longer than 1 character, not a letter, or has already been guessed) and does not penalize or reveal anything, or
 - penalizes the user and updates the number of guesses remaining if the guess is valid and does not exist in the secret word.
4. The game ends when either the user guesses the secret word or the user runs out of guesses.

We will also introduce a feature to make the game easier: the user can input a special 'help' character, `!`, that reveals an unguessed letter at the expense of losing more guesses.

Important: Do NOT change the name, input parameters, or specifications of any of the provided functions! You may add helper functions, but changing the definitions of the given functions will cause the unit tests to fail.

2) Problems (original text preserved)

Problem 1: The Three Helper Functions

To approach the game, we will break down the problem into logical subtasks, creating three helper functions. The starter code in `hangman.py` has implementations for the helper functions `load_words` and `choose_word`. You may read the docstrings associated with each function to understand what they do.

1.1) Determine if the Player has Won (15%)

Implement the function `has_player_won` according to its docstring. This function will be useful in determining when the hangman game has been won (i.e., the user has guessed all the letters in the secret word).

```
>>> secret_word = 'apple'
>>> letters_guessed = ['e', 'i', 'k', 'p', 'r', 's']
>>> print(has_player_won(secret_word, letters_guessed))
False
```

Testing: Run `test_ps2_student.py`. Examine the tests that start with `test_has_player_won`; all four should print `ok`.

1.2) Revealing Letters (15%)

Implement the function `get_word_progress` according to its docstring. This should be fairly similar to `has_player_won`.

Hint: Think about (a) if you need to store information as you loop over a data structure, and (b) how you want to add information to your accumulated result.

```
>>> secret_word = 'apple'
>>> letters_guessed = ['e', 'i', 'k', 'p', 'r', 's']
>>> print(get_word_progress(secret_word, letters_guessed))
*pp*e
```

Testing: Run `test_ps2_student.py`; the four tests starting with `test_get_word_progress` should print `ok`.

1.3) Get Available Letters (15%)

Implement the function `get_available_letters` according to its docstring. This function should return the letters in alphabetical order.

Hint: You might consider using `string.ascii_lowercase`, which is a string comprised of all lowercase letters:

```
>>> import string
>>> print(string.ascii_lowercase)
abcdefghijklmnopqrstuvwxyz
```

```
>>> letters_guessed = ['e', 'i', 'k', 'p', 'r', 's']
>>> print(get_available_letters(letters_guessed))
abcdefghijklmnopqrstuvwxyz
```

Testing: Run `test_ps2_student.py`; the three tests starting with `test_get_available_letters` should print ok.

Problem 2: The Game (55%)

Implement the `hangman` function, which takes two parameters: (1) `secret_word`, the secret word the user is to guess; and (2) `with_help`, a boolean representing whether or not the game is to be played with the 'help' functionality.

Calling the `hangman` function starts up an interactive game between the user and the computer. Be sure you take advantage of the three helper functions `has_player_won`, `get_word_progress`, and `get_available_letters` from Problem 1! You may write additional helper functions if you need them.

To test your `hangman` function, call it from within the `if __name__ == "__main__":` condition at the bottom of `hangman.py`. Manually set the secret word to make it easier to test your code. In the end, you should also test your program with **the secret word selected at random**.

```
if __name__ == "__main__":
    secret_word = "tact"
    with_help = False
    hangman(secret_word, with_help)
```

Important: Make your print statements as close to the example games in the appendix of the original handout as possible — the autograder compares printed output line by line.

2.1) Game Setup

1. `secret_word` along with the boolean `with_help` are passed into the `hangman` function as parameters.
2. At the start of the game, display how many letters `secret_word` contains.
3. Users start with **10 guesses**.

2.2) User-computer interaction

1. Before each guess, you should display to the user:
 - At least **three (3)** dashes (e.g. `-----`) to separate individual guesses from each other.
Leaving out the row of dashes will cause the tester to fail.
 - How many guesses they have remaining
 - All the letters that have not yet been guessed

2. Ask the user to supply one guess at a time.
 - The user can type any number, symbol, or letter. Your code should only accept capital and lowercase single letters as valid guesses!
 - If the game is played with help, your code should also accept the help character (!).
3. Immediately after each guess, you should display:
 - Whether or not the letter is in the secret word
 - The word with guessed letters revealed and unguessed letters as asterisks (*)

Example round (user input shown after the prompt):

```
-----
You have 10 guesses left.
Available letters: abcdefghijklmnopqrstuvwxyz
Please guess a letter: a
Good guess: *a**
-----
You have 10 guesses left.
Available letters: bcdefghijklmnopqrstuvwxyz
Please guess a letter: b
Oops! That letter is not in my word: *a**
-----
You have 9 guesses left.
Available letters: cdefghijklmnopqrstuvwxyz
Please guess a letter: 2
Oops! That is not a valid letter. Please input a letter from
the alphabet: *a**
```

Hints:

1. Use calls to the `input()` function to get the user's guess, and check that the input is an alphabet letter (or the help character if playing with help).
2. Since the words in `words.txt` are lowercase, we suggest converting user input to lowercase.
3. You may find the string functions `str.isalpha()` and `str.lower()` helpful.

2.3) Guesses Remaining

If the user inputs:

1. Anything besides a letter in the alphabet (e.g. symbols or numbers), tell the user that they can only input an alphabet letter. The user loses **no** guesses. (Note: when the game is played with help, `!` is also a valid input.)
2. A letter that has already been guessed: print a message telling the user the letter has already been guessed before. The user loses **no** guesses.
3. Any letter that hasn't been guessed before and the letter is in the secret word: the user loses **no** guesses.
4. **Consonants:** if the user inputs a consonant that hasn't been guessed and the consonant is not in the secret word, the user loses **one** guess.
5. **Vowels:** if the user inputs a vowel that hasn't been guessed and the vowel is not in the secret word, the user loses **two** guesses. Vowels are `a`, `e`, `i`, `o`, and `u`. The letter `y` does not count as a vowel.

(Note: if a user inputs an incorrect vowel that hasn't been guessed and there is only one guess remaining, the user loses and the game is over.)

2.4) The Game with Help

- If you type the special character `!`, the computer will provide you with one of the missing letters in the secret word at a cost of **three** guesses. This character should be the **only** non-letter input that your game accepts as a guess.
- **If you do not have at least three guesses remaining**, the computer will warn you of this and let you try again. You lose **no** guesses.

Note: the user can play with this feature only when the `with_help` parameter is `True`.

As a starting point, we suggest writing a **helper function that chooses a letter to reveal**. It should take two arguments: the secret word and the string of available letters (from `get_available_letters`). This helper function should create a string `choose_from` containing the unique letters that are in both the secret word and the available letters. You can then use the following statements to pick a random character `revealed_letter` from that string:

```
new = random.randint(0, len(choose_from)-1)
revealed_letter = choose_from[new]
```

Your helper function should then return this `revealed_letter`. Back in your original game logic, you'll need to add a conditional statement to catch the case of the user inputting `!`. This case, if triggered, can add the letter returned by your helper function to `letters_guessed`, show the new word progress, decrement the remaining guesses by 3, and continue the gameplay.

Example (help mode):

```
-----
You currently have 10 guesses left
Available letters: abcdefghijklmnopqrstu
Please guess a letter: !
Letter revealed: r
r*****r
-----
You currently have 1 guess left
Available letters: abdefghijklmnopqstu
Please guess a letter: !
Oops! Not enough guesses left: r*e*ar
```

2.5) Game Termination

1. The game ends when the user guesses all the letters in `secret_word` or has 0 guesses remaining.
2. If the user wins, print a congratulatory message, and tell the user their score:

```
total_score = (guesses_remaining + 4 × number of unique letters in secret_word) + (3
× length of secret_word)
```

Example: for a game with secret word "asleep" with 6 guesses remaining, there are a total of 5 unique letters (a, s, l, e, p). Thus, the final score is $(6 + 4 \times 5) + (3 \times 6) = 44$.

3. If the player runs out of guesses before completing the word, tell them they lost and reveal the word to the user when the game ends.

Example endings:

```
-----  
Congratulations, you won!  
Your total score for this game is: 54
```

```
-----  
Sorry, you ran out of guesses. The word was else.
```

2.6) Testing your code

Look carefully at the example hangman games in the appendix of the original OCW handout and make your print statements as close to the example games as possible. If you scroll to the bottom of `hangman.py`, uncomment the bottom three lines to choose a random secret word and play hangman:

```
if __name__ == "__main__":  
    # secret_word = choose_word(wordlist)  
    # with_help = False  
    # hangman(secret_word, with_help)
```

Student Tester. Run `test_ps2_student.py`. If your function is correct, the tests `test_play_game_short`, `test_play_game_short_fail`, and `test_play_game_with_help` should print `ok`.

Debugging tips (from the original handout):

- The autograder tests your printed output against the expected printed output, so your output needs to have **the same number of lines** as the expected output, including the dashes at the end of a turn.
- If you fail any `test_play_game` tests, a file called `run_game_test_results.txt` appears next to the tester with the expected and actual output. Input prompts do not show up in that file — this is normal.
- Provide a **non-empty string parameter to input**, i.e. call `input("Please guess a letter: ")`, not `input("")` — the tester cannot feed input otherwise.
- `print('foo', 'bar')` inserts a space between the strings; the optional `sep` and `end` parameters control separators and line endings.

3) Deliverables

Submit to the course LMS:

1. **hangman.py** — save your solutions with the original file name. **Do not save your file with a different name!**
2. At the start of the file, in comments: name, collaborators, time spent.

```
# Problem Set 2, hangman.py
# Name: Hong Gildong
# Collaborators: ...
# Time Spent: ...
```

3. A **capture of the `test_ps2_student.py` output** (all tests `ok`).
4. A **transcript of one complete game you played with `with_help = True`** (copy the console text into a `.txt` file), plus a 3–5 line comment in `hangman.py` explaining how you decomposed the game loop into helper functions.

Collaboration. Students may work together, but each student should write up and hand in their assignment separately. Students may not submit the exact same code, and are not permitted to look at or copy each other's code or code structure.

4) Grading Rubric Summary (course common rubric)

Criterion	Weight	Applied to PS2 as
Correctness & edge cases	45%	Unit tests for 3 helpers + game-output tests (invalid input, repeated letter, vowel penalty, <code>!</code> with <code><3</code> guesses, win/lose endings, score formula)
Problem decomposition	20%	Three helper functions used inside <code>hangman</code> ; additional helpers (e.g. reveal-letter chooser) where appropriate
Student-written tests	15%	Tester capture + played-game transcript with help mode
Explanation	10%	Decomposition comment block; docstrings preserved
Code clarity & style	10%	Naming, no duplicated logic between help/no-help paths

5) [Instructor Checklist]

Changes from the original (MIT 6.100L F22 PS2):

1. MIT-specific logistics removed: Pset Buddy notice, half-way submission (Oct 12) / final submission (Oct 19) dates, MIT upload boxes, Piazza references. Replaced by course LMS deliverables (§3).
2. The two full-length appendix game transcripts (win/lose/help examples, original §4) are not duplicated in this packet; students are directed to the original OCW handout PDF (distributed alongside) for line-by-line output matching. Representative excerpts are kept in §2.2/§2.4/§2.5.
3. Deliverable 4 (game transcript + decomposition comment) added to support the course rubric; adds no new problem content.
4. Game rules, guess penalties (consonant –1, vowel –2, help –3), 10 starting guesses, score formula, function names/parameters, all example usages and tester expectations — **preserved unchanged**.

Problem count: original 4 (1.1, 1.2, 1.3, 2-game) → packet 4 (equal).

Prerequisite check: ☒ Confirmed — no items beyond Week 4 prerequisites (functions, strings, lists, loops, conditionals, `random` as a black box; no dictionaries or classes required).